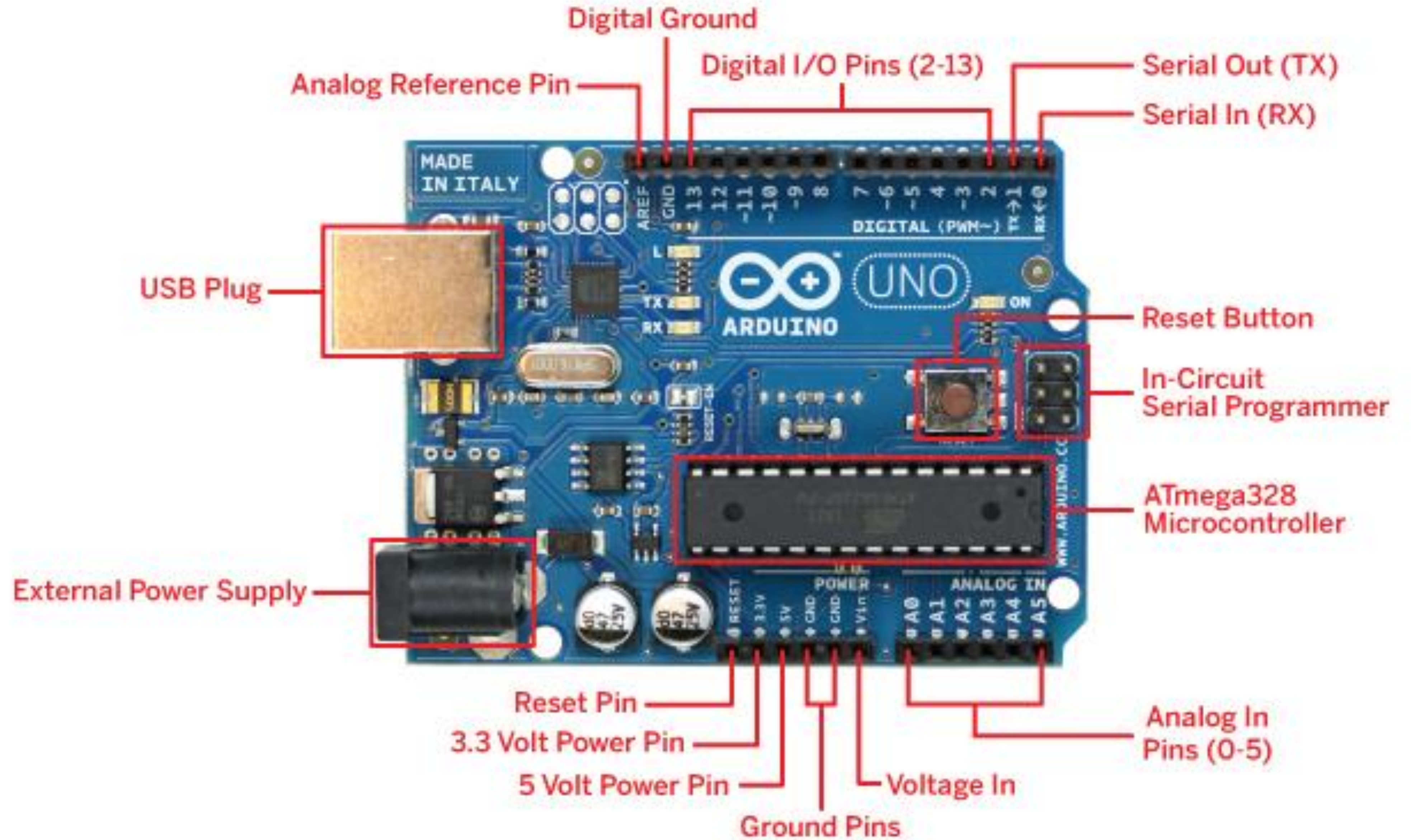
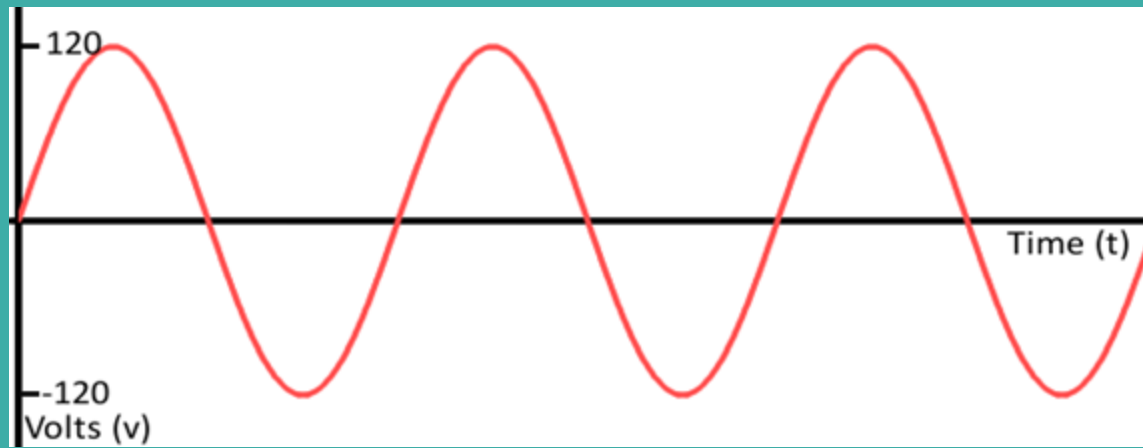


Arduino Analog & Sensing!

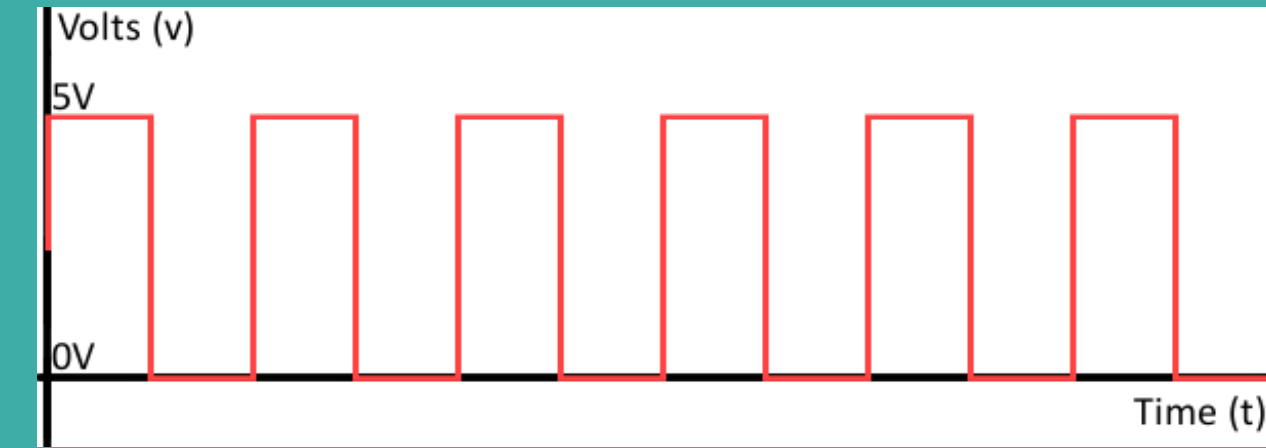
RECAP



ANALOG



DIGITAL



Signal Analog signal is a continuous signal which represents physical measurements.

Digital signals are discrete time signals generated by digital modulation.

Waves Denoted by sine waves

Denoted by square waves

Representation Uses continuous range of values to represent information

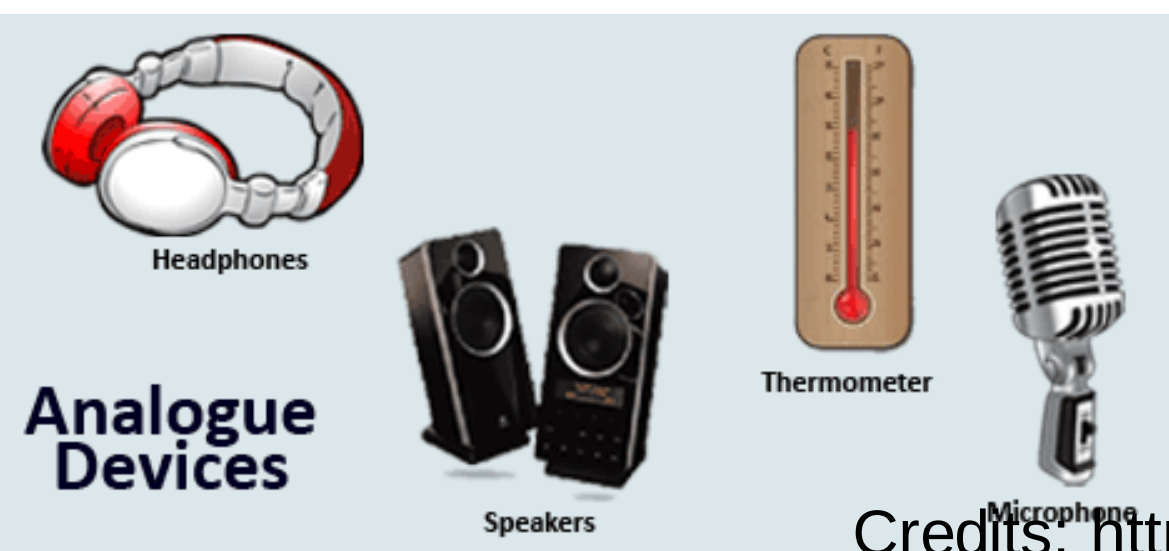
Uses discrete or discontinuous values to represent information

Example Human voice in air, analog electronic devices.

Computers, CDs, DVDs, and other digital electronic devices.

Technology Analog technology records waveforms as they are.

Samples analog waveforms into a limited set of numbers and records them.



Digital pins



On board 14 Digital Pins are present

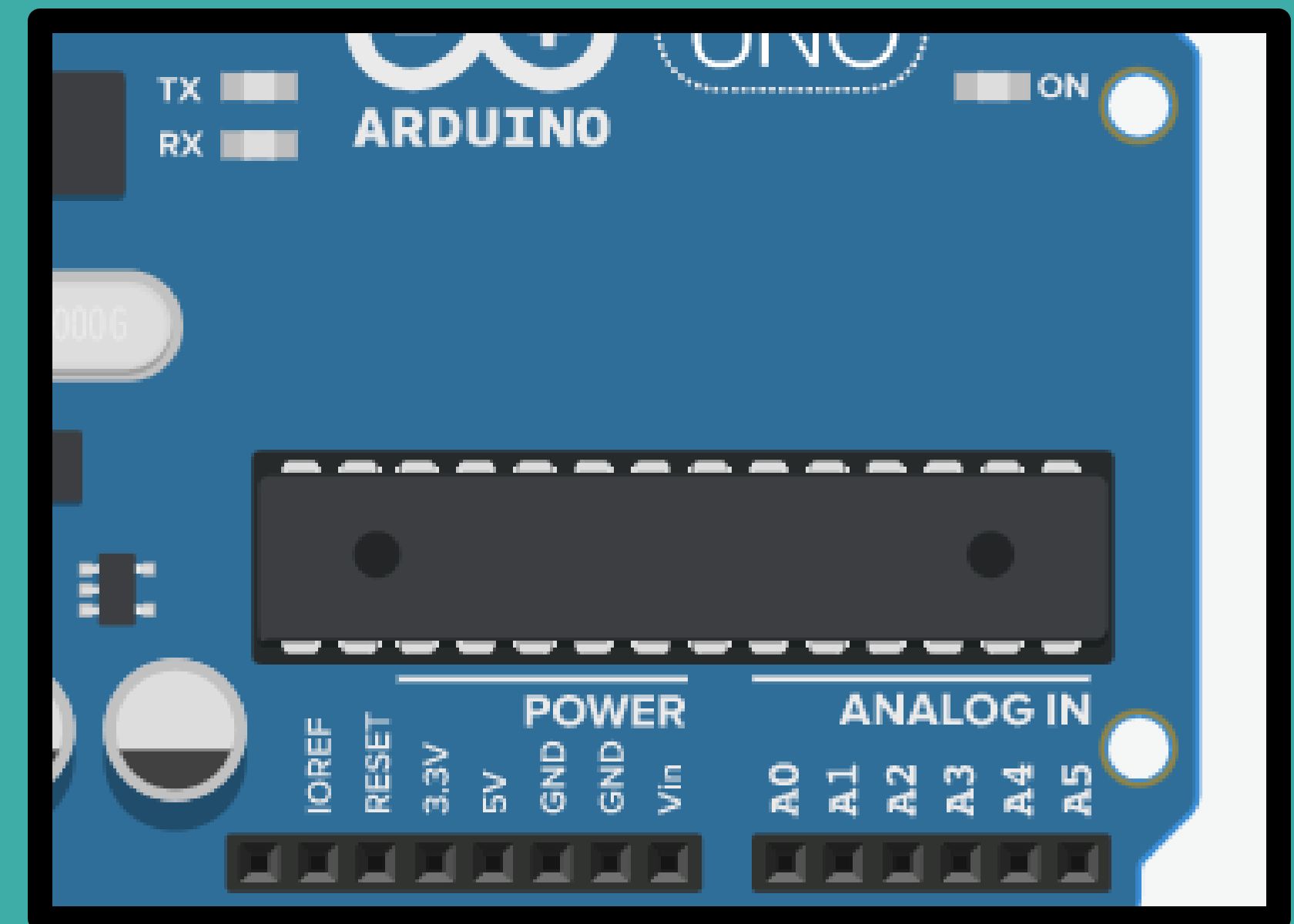
Generally, provides only two kinds of output i.e.

High = 5 Volt = 1

Low = 0 Volt = 0

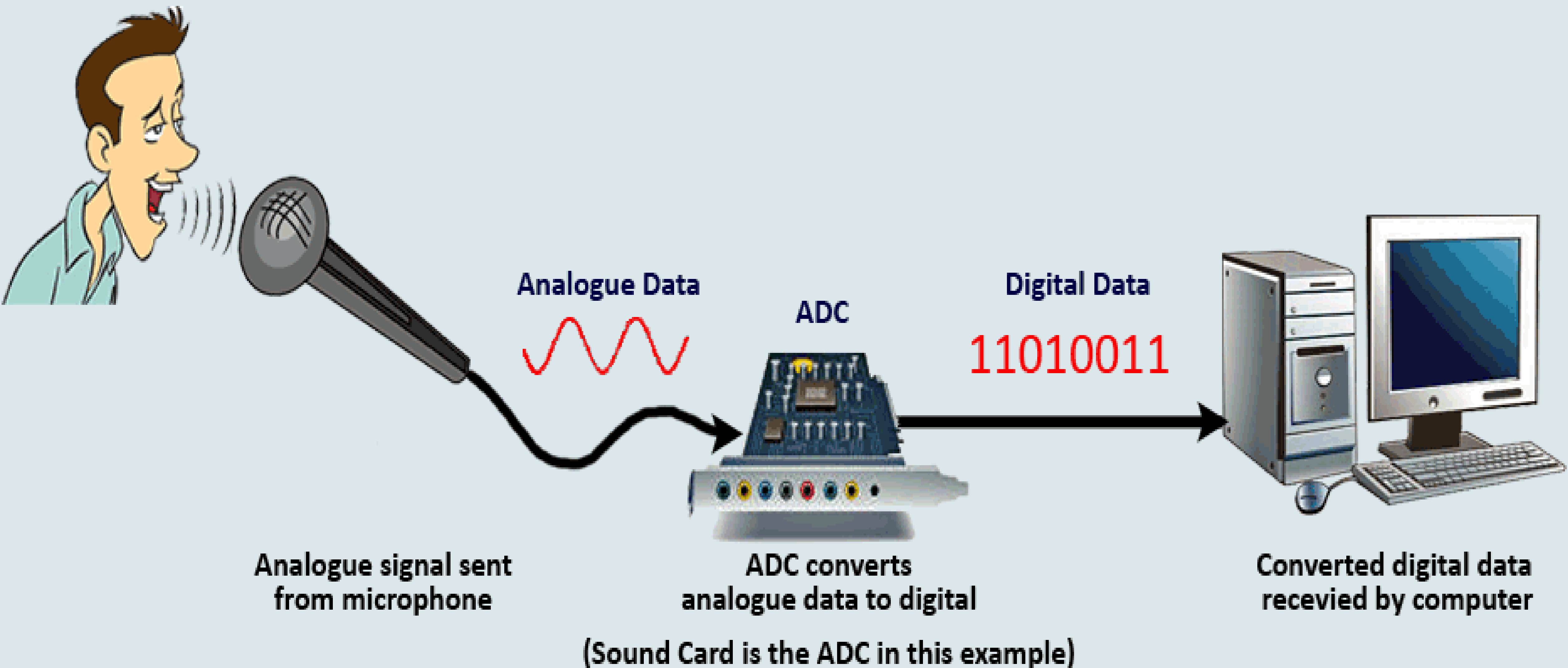
Onboard 6 channel analog-to-digital (A/D) converter.

ARef: On this pin, user defined reference value of ADC is given (Not more than 5 Volt).



Analog pins

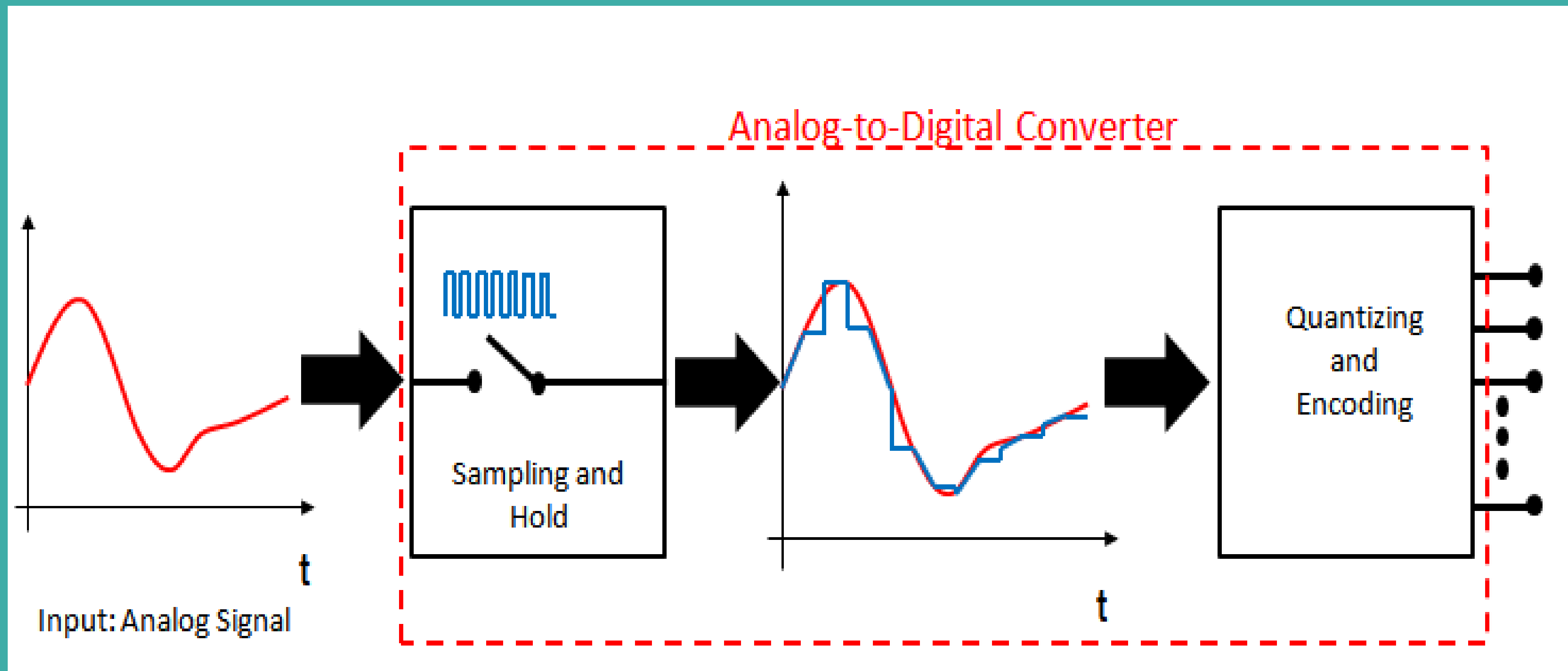
Analog to Digital Converter(ADC)



ADC process

Two main steps of the process

1. Sampling and Holding
2. Quantization and Encoding



Quantizing

The number of possible states that the converter can output is: $N=2^n$ (n is the number of **bits** in the AD converter)

Example: For a 3 bit A/D converter, $N=2^3=8$.

Analog quantization size:

$$Q=(V_{\max}-V_{\min})/N=(10\text{ V}-0\text{ V})/8=1.25\text{V}$$

Discrete Voltage	Ranges(V)
0	0.00-1.25
1	1.25-2.50
2	2.50-3.75
3	3.75-5.00
4	5.00-6.25
5	6.25-7.50
6	7.50-8.75
7	8.75-10.00

Encoding

Here we **assign the digital value** (binary number) to each state for the computer to read.

Output States Output Binary Equivalent

0 000

1 001

2 010

3 011

4 100

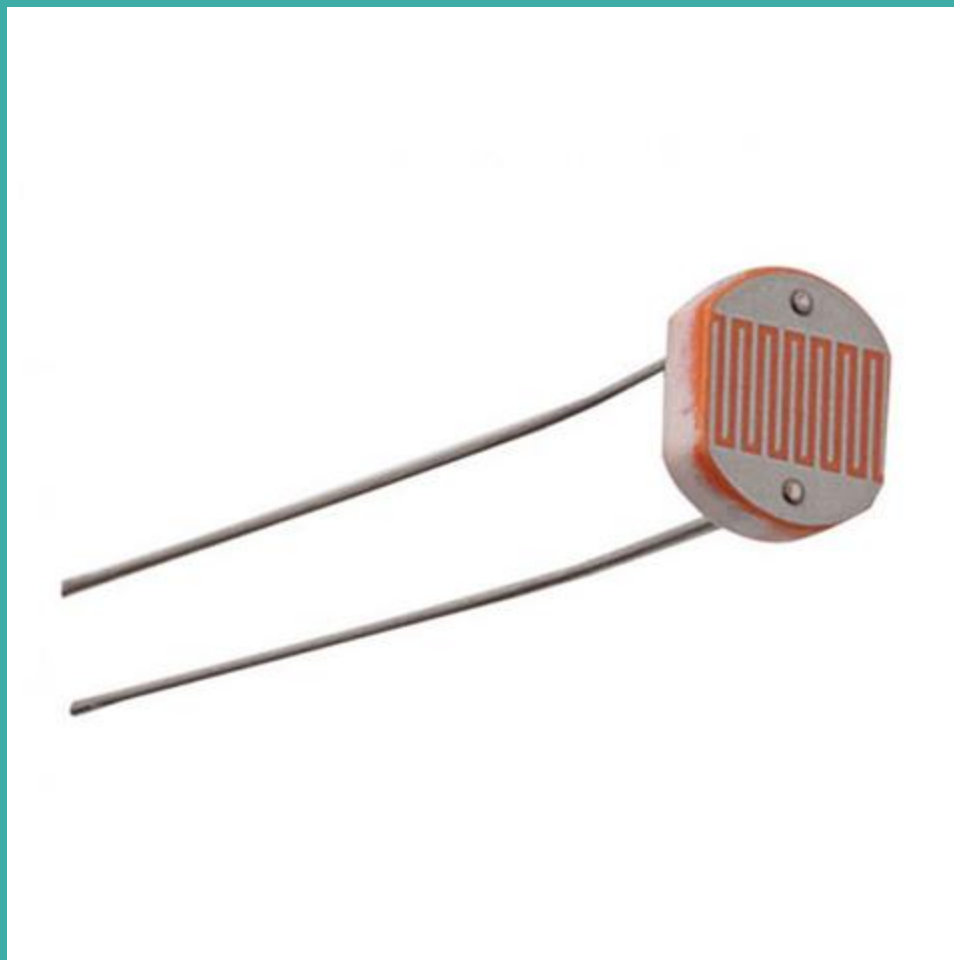
5 101

6 110

7 111

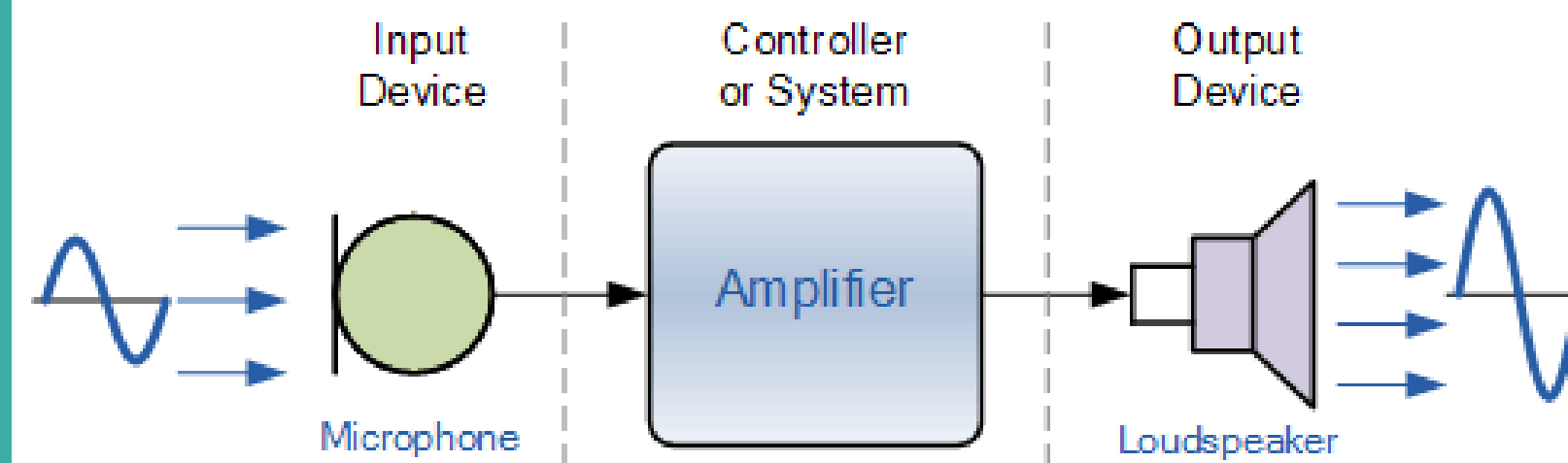
Sensing

Detects and responds to some type of input from the physical environment



Transduction

Converting one form of energy to another

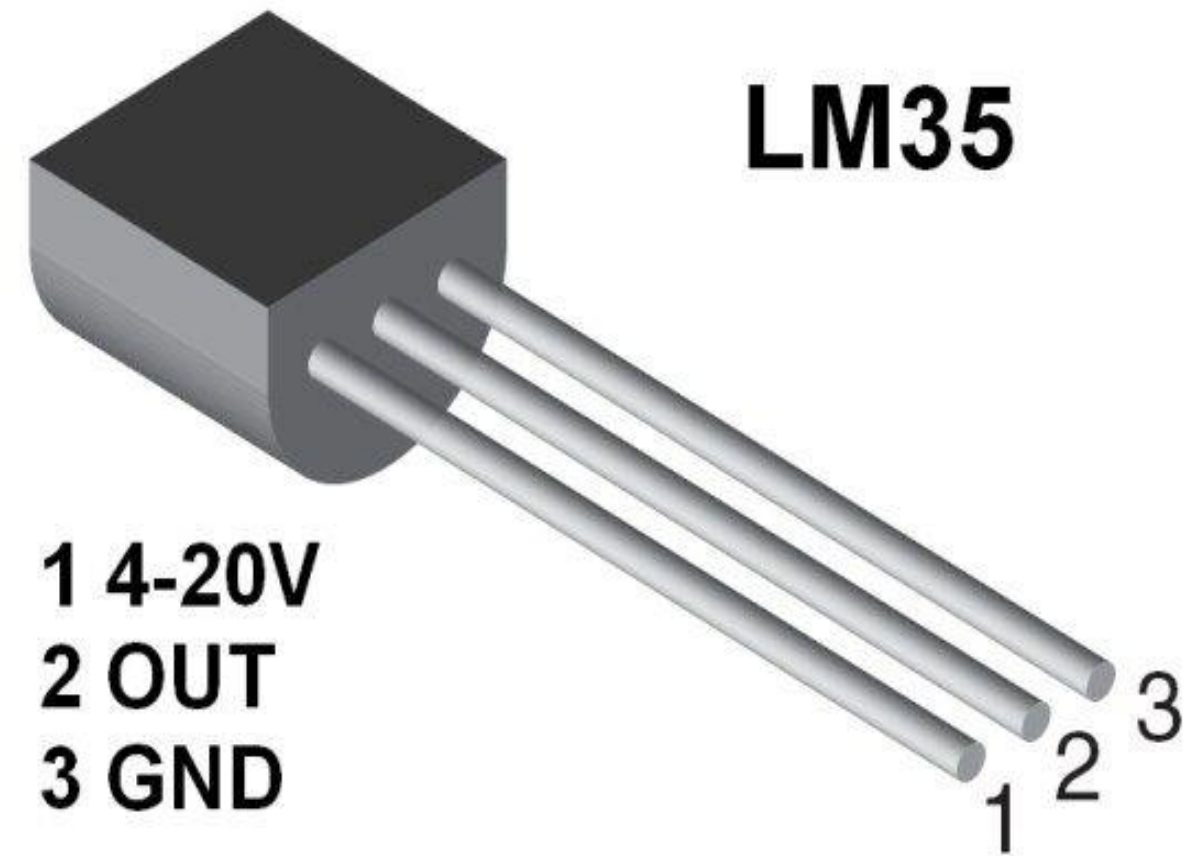


Actuation

To put into motion or action

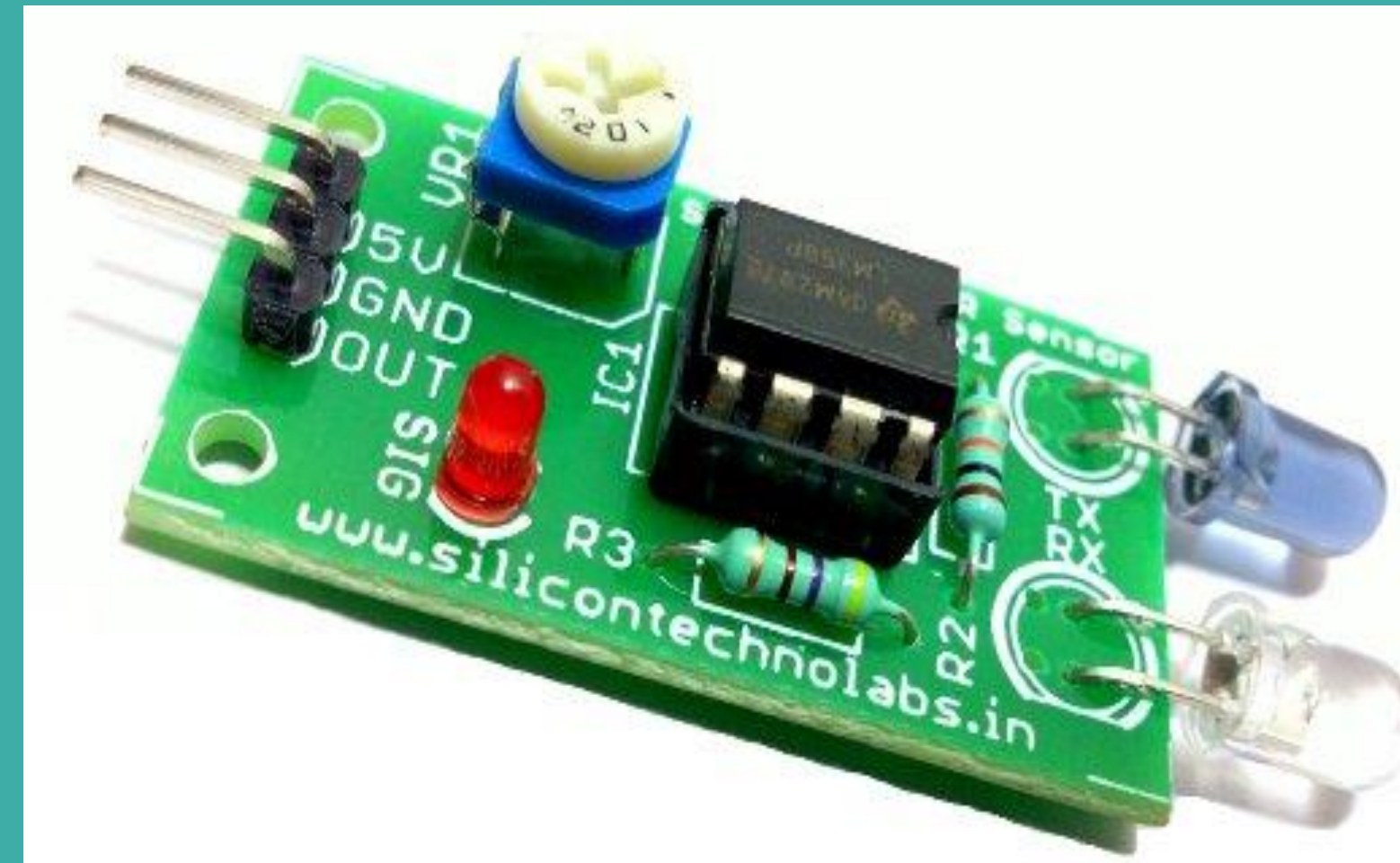
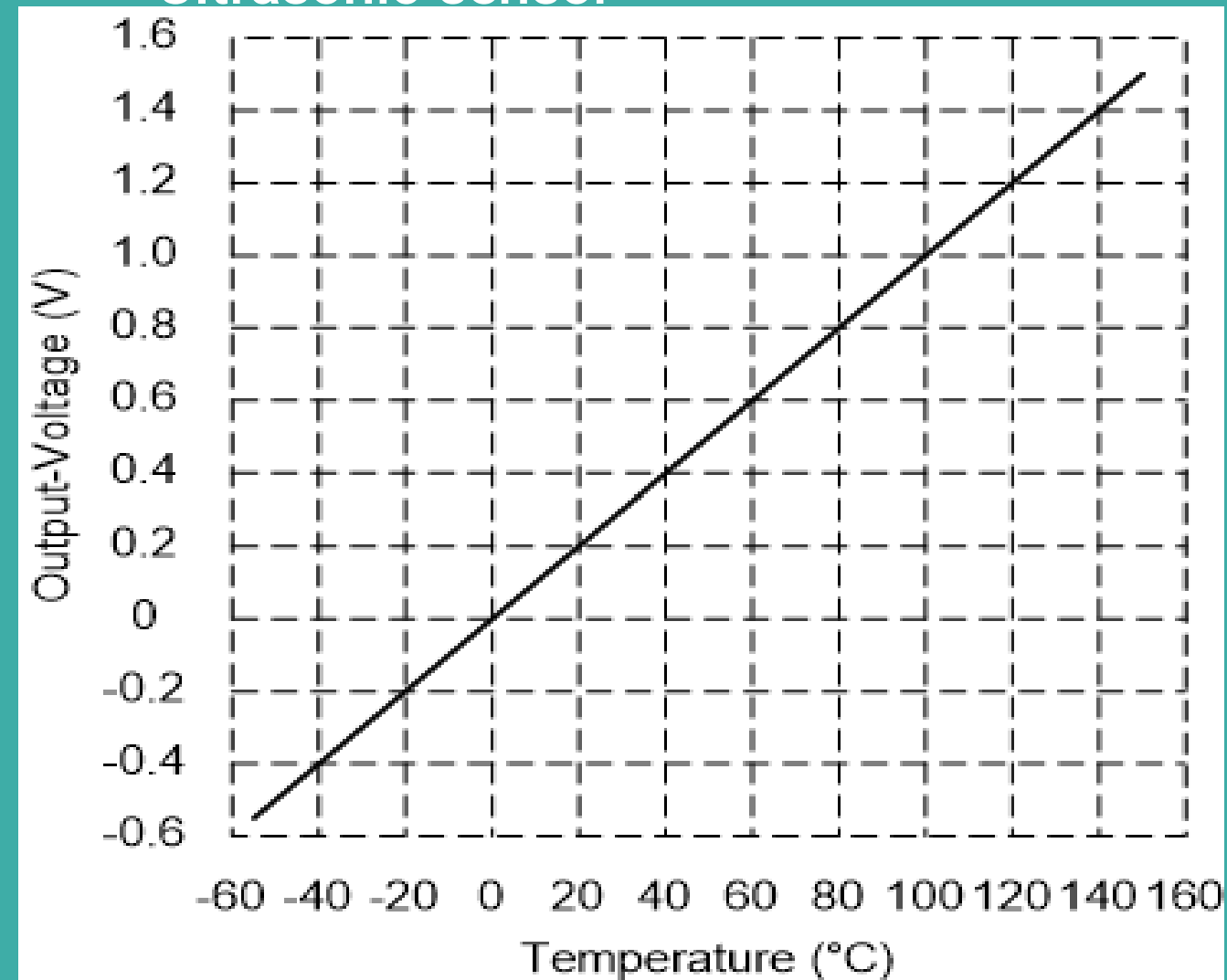


Types of Sensors

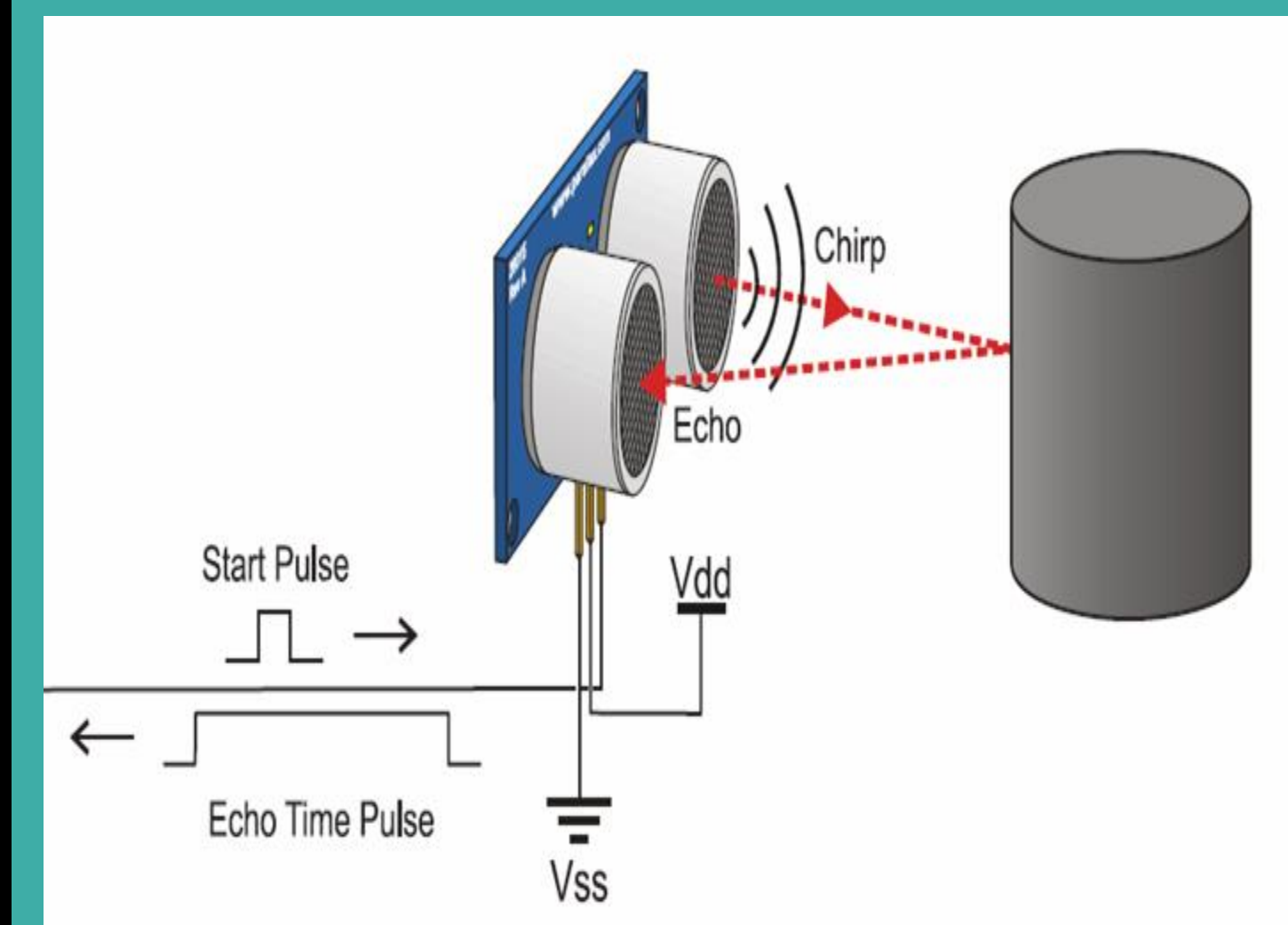
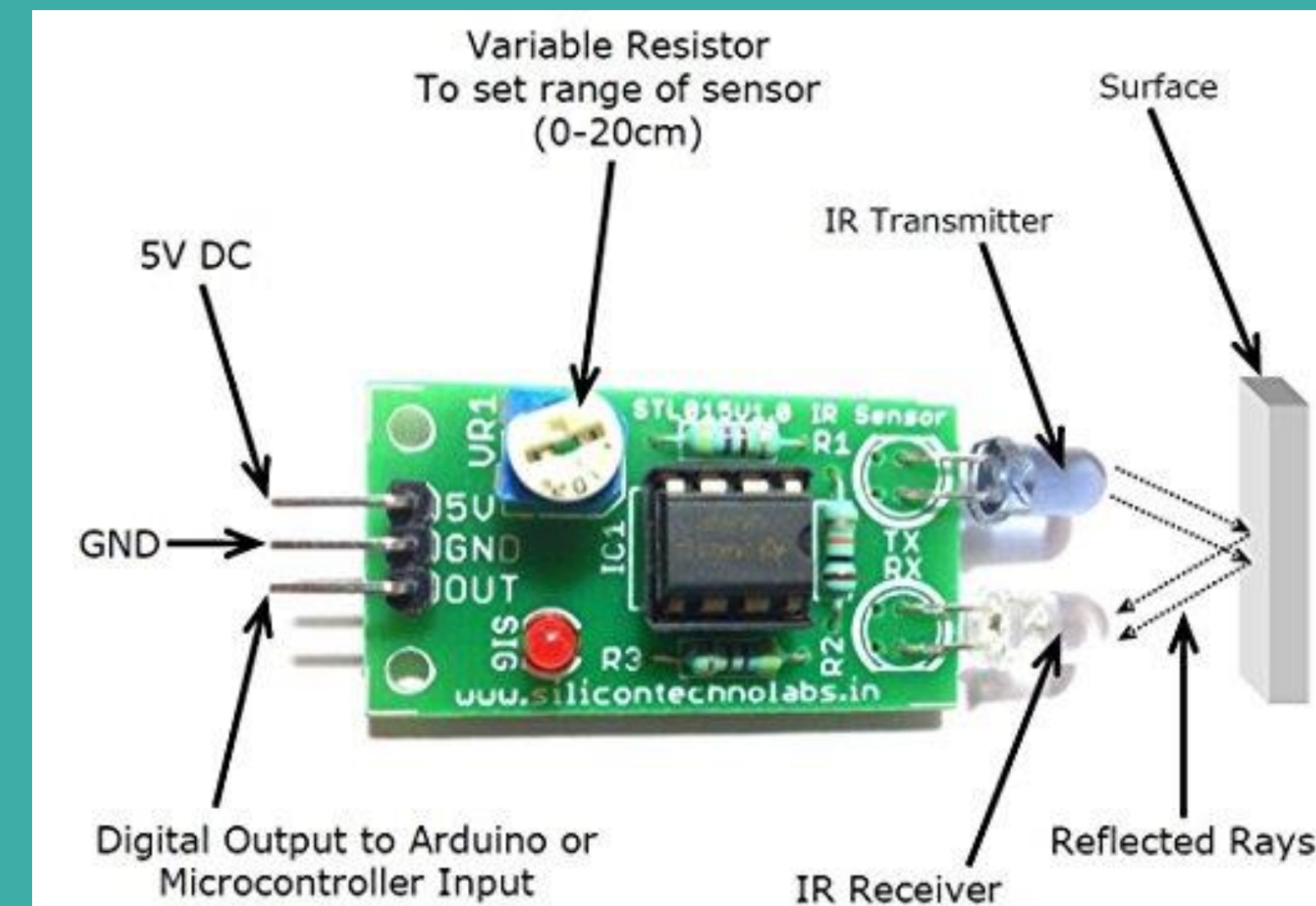


Temperature sensor(LM35)

Ultrasonic sensor



IR Sensor



Count the sensors



Classifications of Sensing

Direct and hybrid transduction

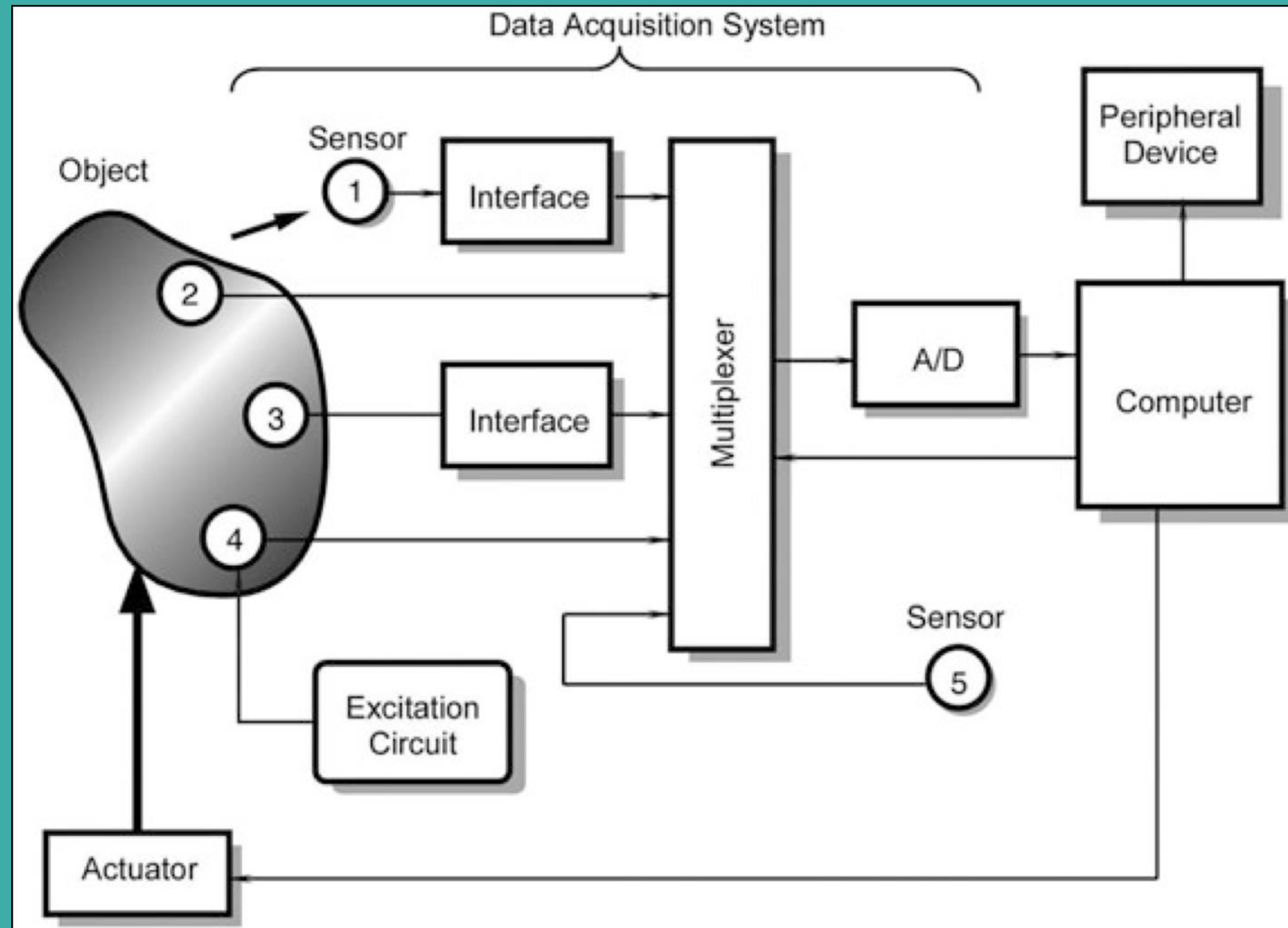
Absolute and relative

Active and passive

Intrinsic and extrinsic

Contact and noncontact

Data Acquisition System



Analog Commands on Arduino

SENSING

- `digitalRead()` works on all pins. It will just round the analog value received and present it to you. If `analogRead(A0)` is greater than or equal to 512, `digitalRead(A0)` will be 1, else 0.
- `analogRead()` works only on analog pins. It can take any value between 0 and 1023.

ACTUATION

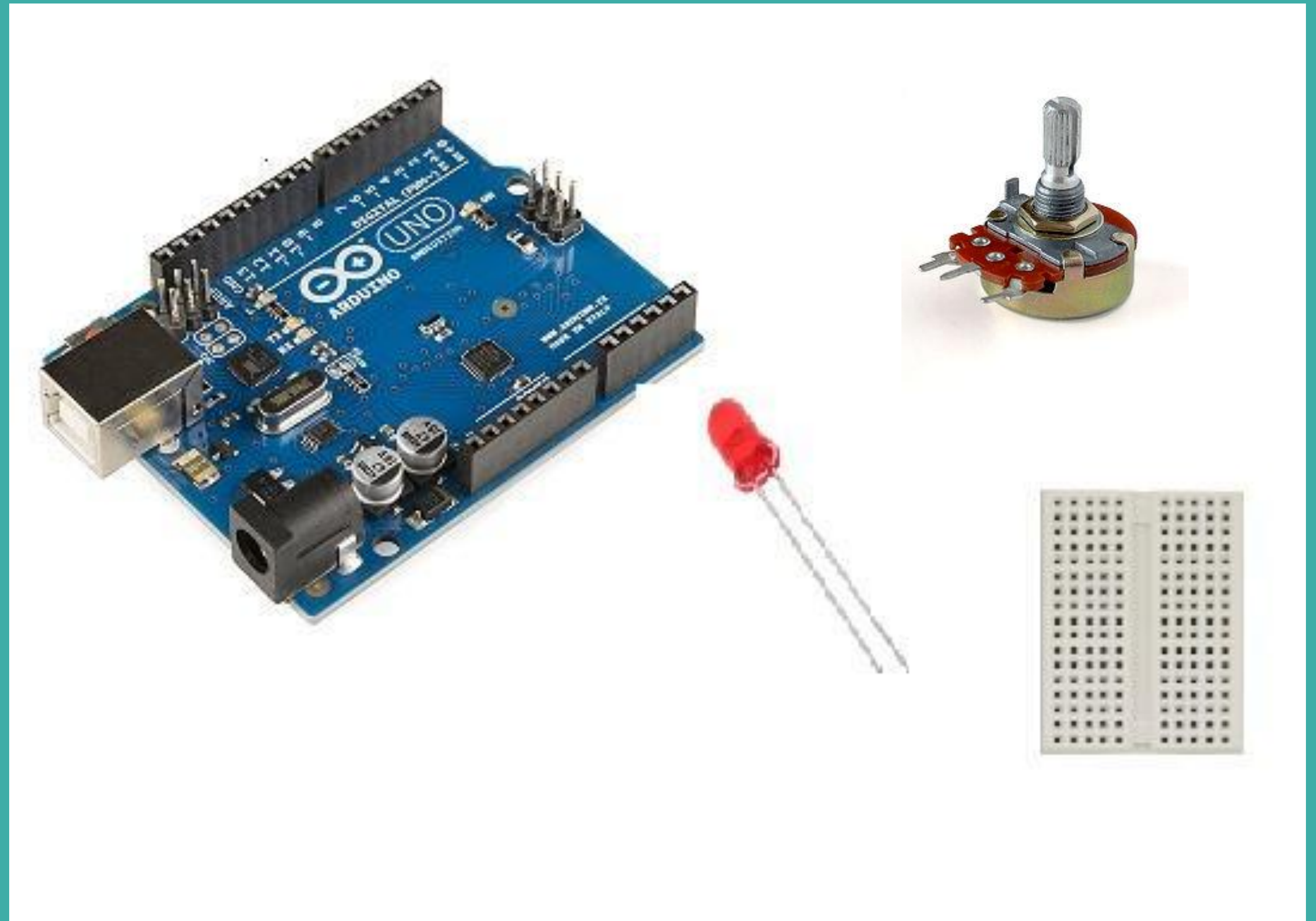
- `digitalWrite()` works on all pins, with allowed parameter 0 or 1. `digitalWrite(A0,0)` is the same as `analogWrite(A0,0)`, and `digitalWrite(A0,1)` is the same as `analogWrite(A0,255)`
- `analogWrite()` works on all analog pins and all digital [PWM](#) pins. You can supply it any value between 0 and 255.

Activity-1

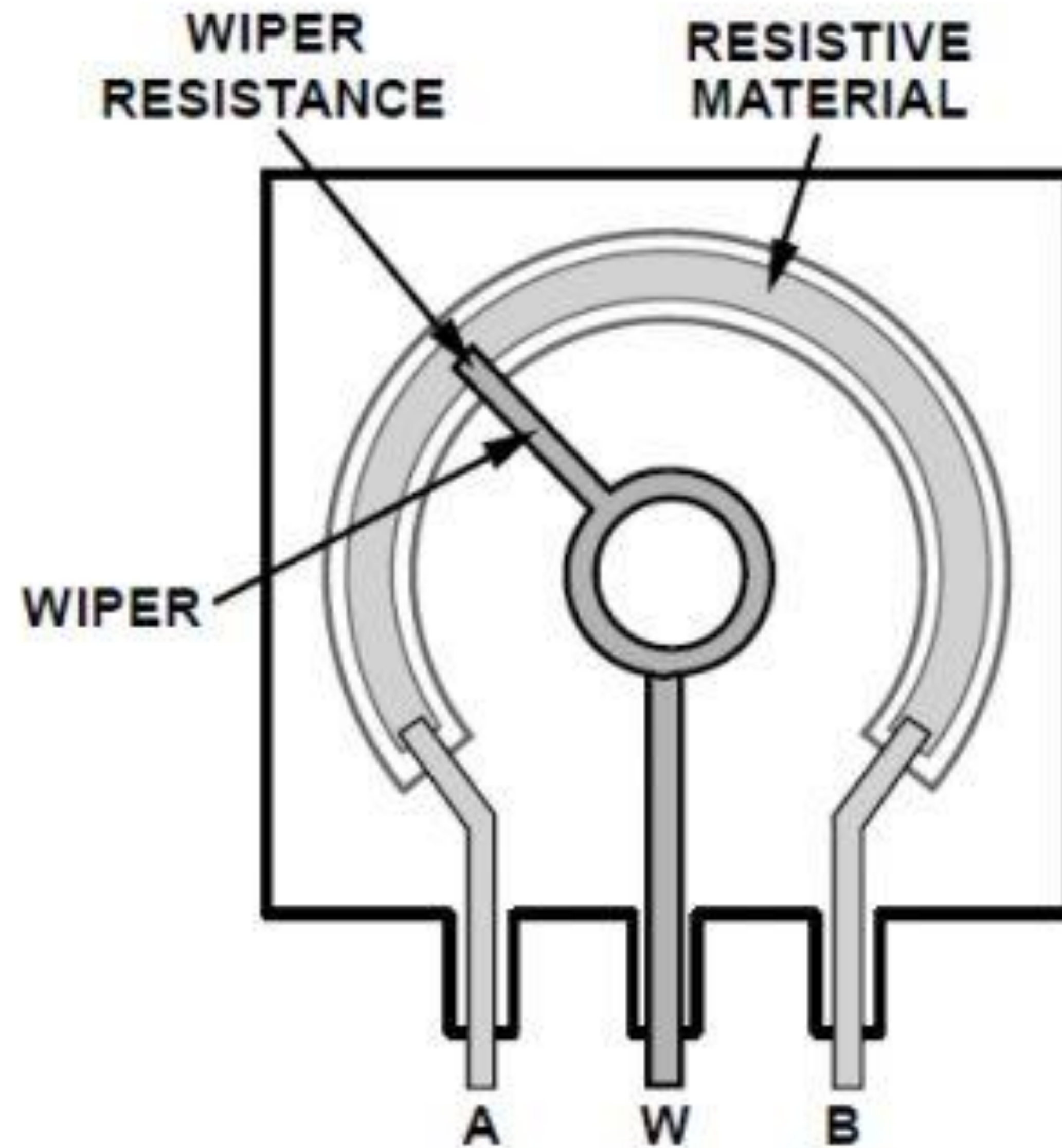
Control LED blinking using Potentiometer

Step 1: What You Will Need

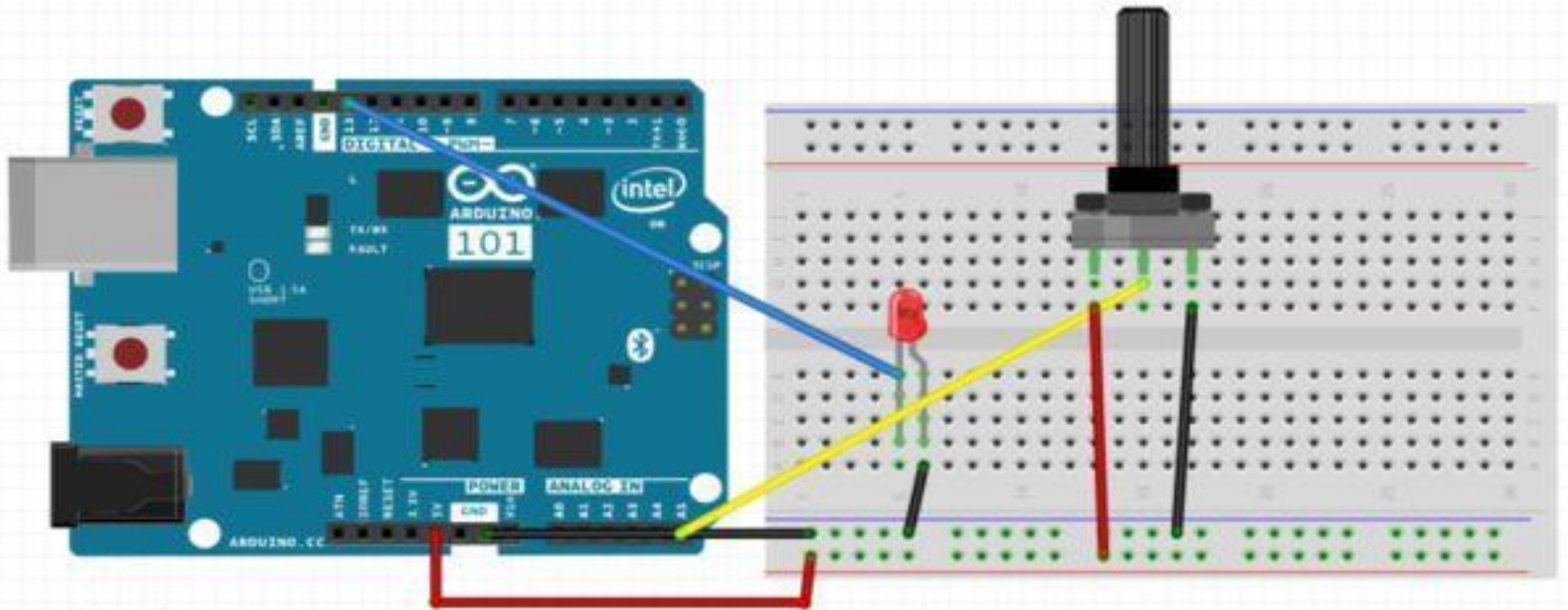
- 1.Arduino Uno
- 2.Potentiometer
- 3.Breadboard
- 4.LED
- 5.Connecting wires



Potentiometer Black Box

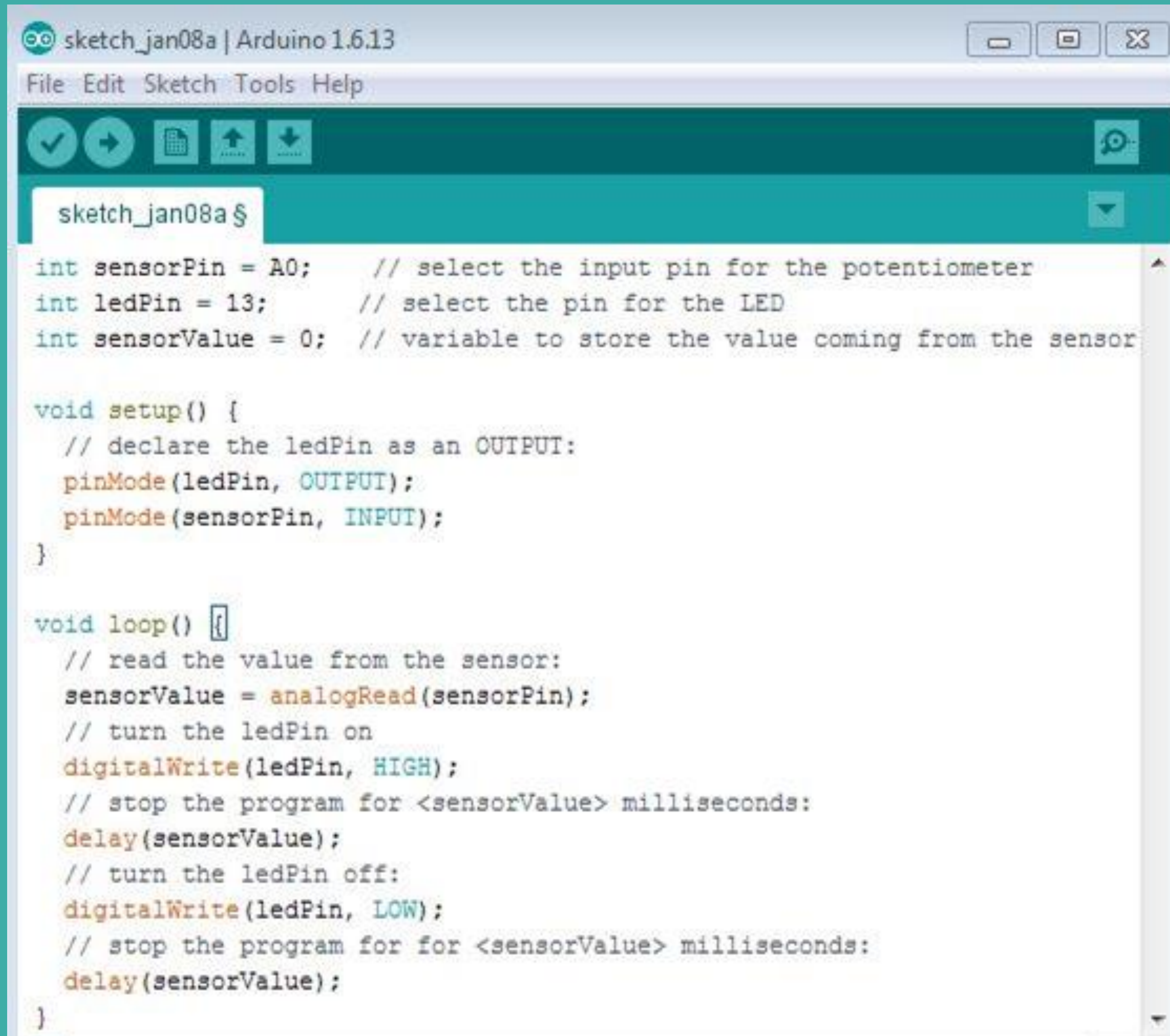


Step 2: Circuit



fritzing

Step 3: IDE Code

A screenshot of the Arduino IDE interface. The title bar shows 'sketch_jan08a | Arduino 1.6.13'. The menu bar includes 'File', 'Edit', 'Sketch', 'Tools', and 'Help'. The toolbar contains icons for checking, running, saving, and uploading. The main text area shows the following code:

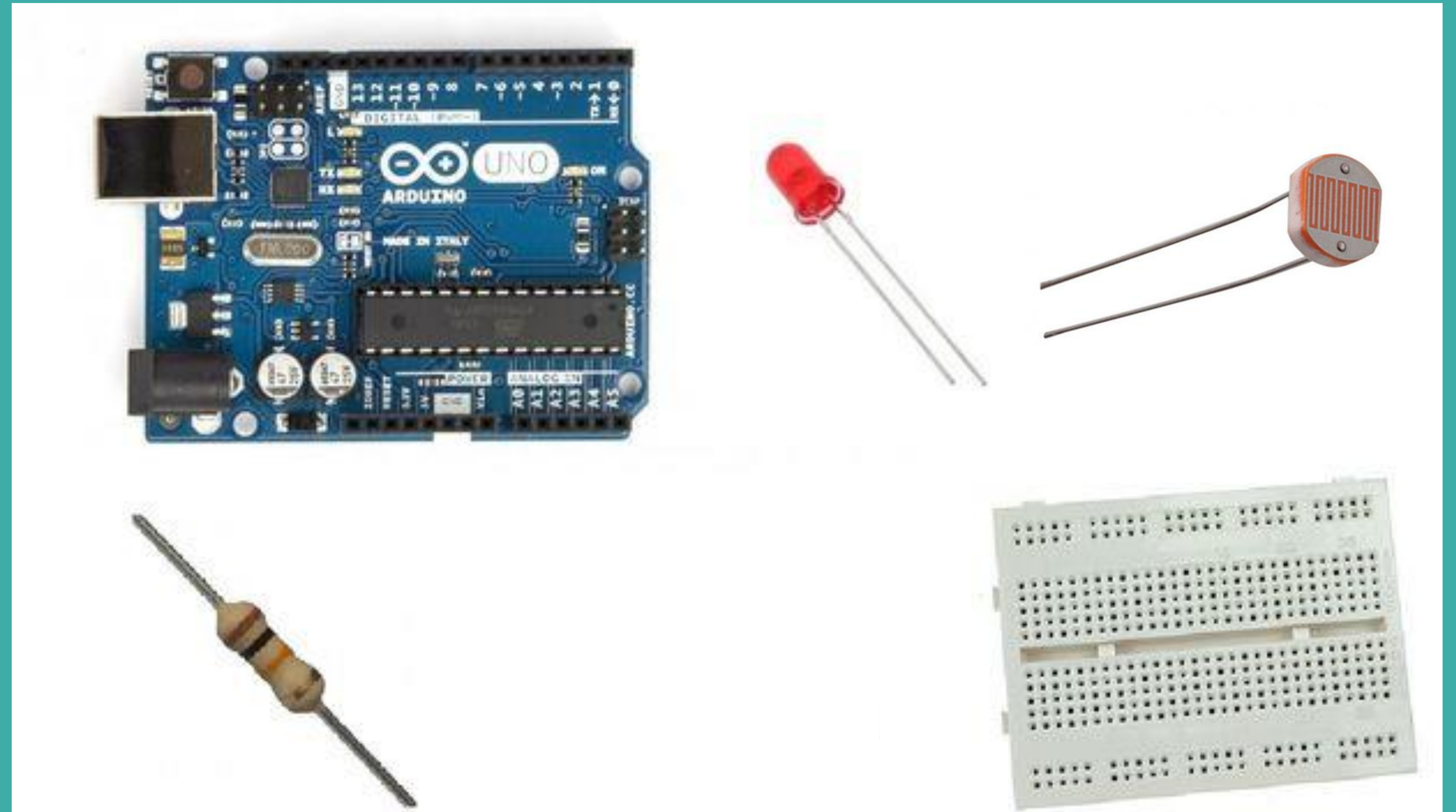
```
sketch_jan08a $  
  
int sensorPin = A0;    // select the input pin for the potentiometer  
int ledPin = 13;       // select the pin for the LED  
int sensorValue = 0;   // variable to store the value coming from the sensor  
  
void setup() {  
  // declare the ledPin as an OUTPUT:  
  pinMode(ledPin, OUTPUT);  
  pinMode(sensorPin, INPUT);  
}  
  
void loop() {  
  // read the value from the sensor:  
  sensorValue = analogRead(sensorPin);  
  // turn the ledPin on  
  digitalWrite(ledPin, HIGH);  
  // stop the program for <sensorValue> milliseconds:  
  delay(sensorValue);  
  // turn the ledPin off:  
  digitalWrite(ledPin, LOW);  
  // stop the program for for <sensorValue> milliseconds:  
  delay(sensorValue);  
}
```

Activity-2

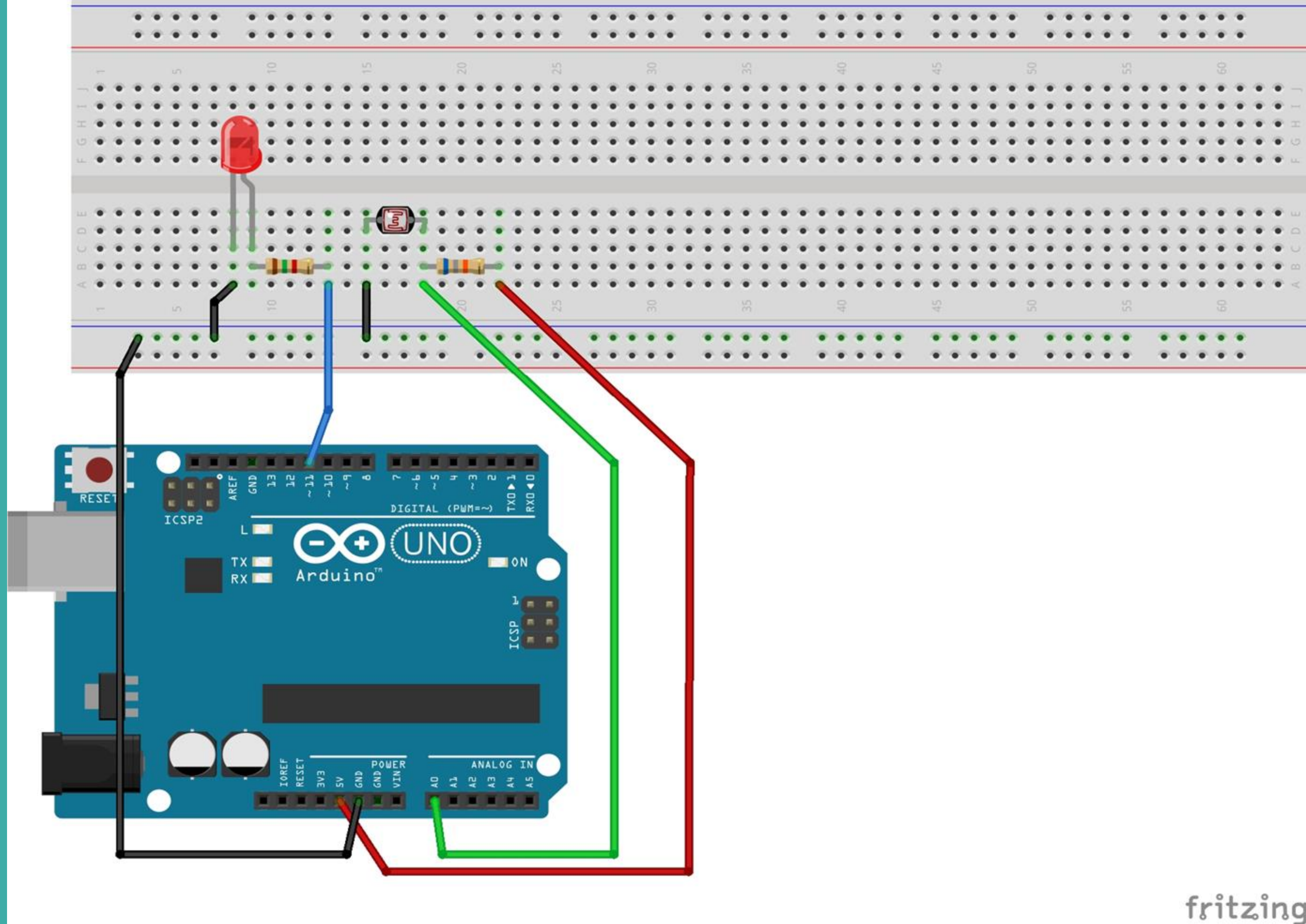
Control LED with a Light-Dependent Resistor (LDR)

Step 1: What You Will Need

1. An Arduino Uno
2. LDR
3. Breadboard
4. 10K resistor
5. Connecting wires
6. LED
7. 220 Ohm Resistor



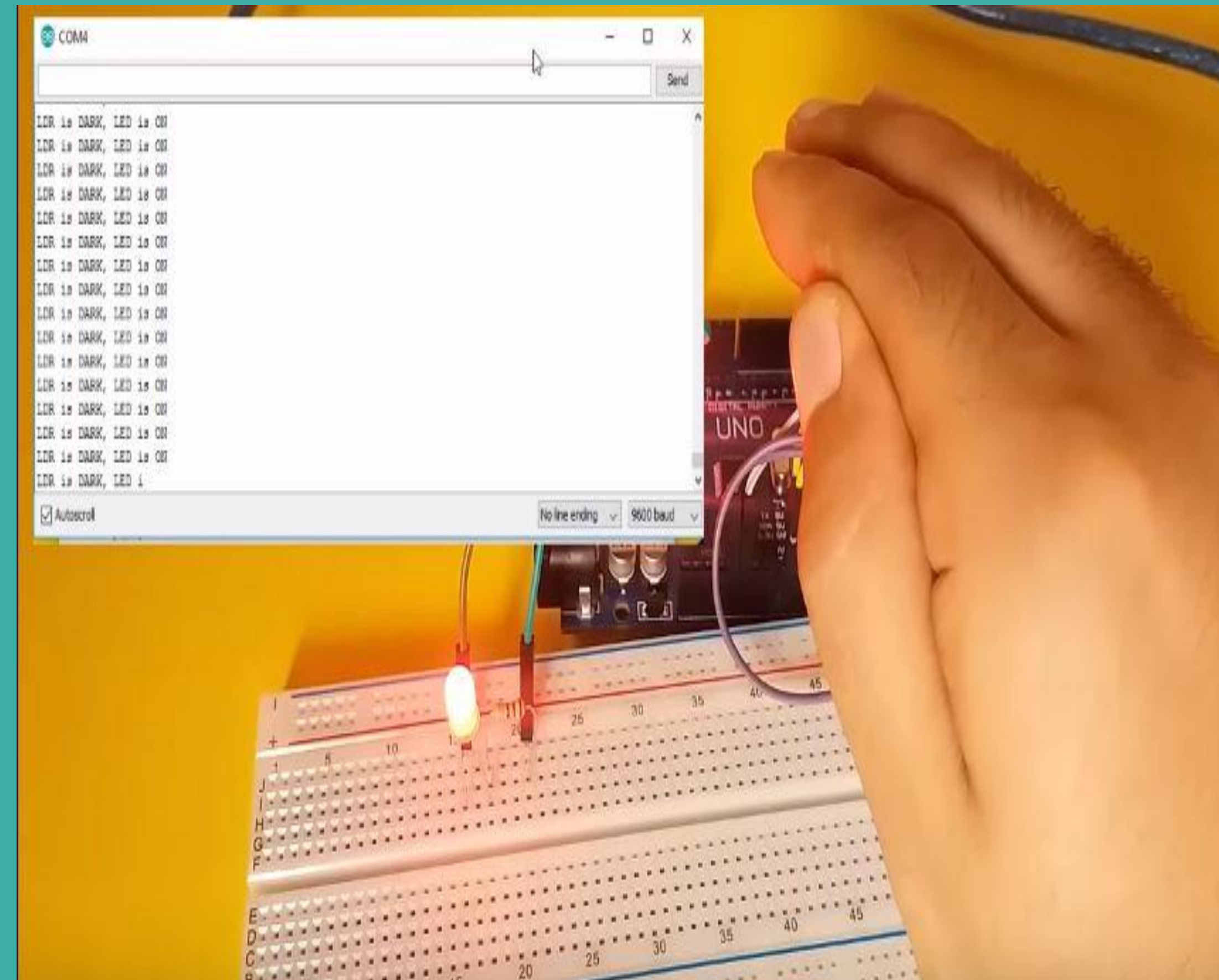
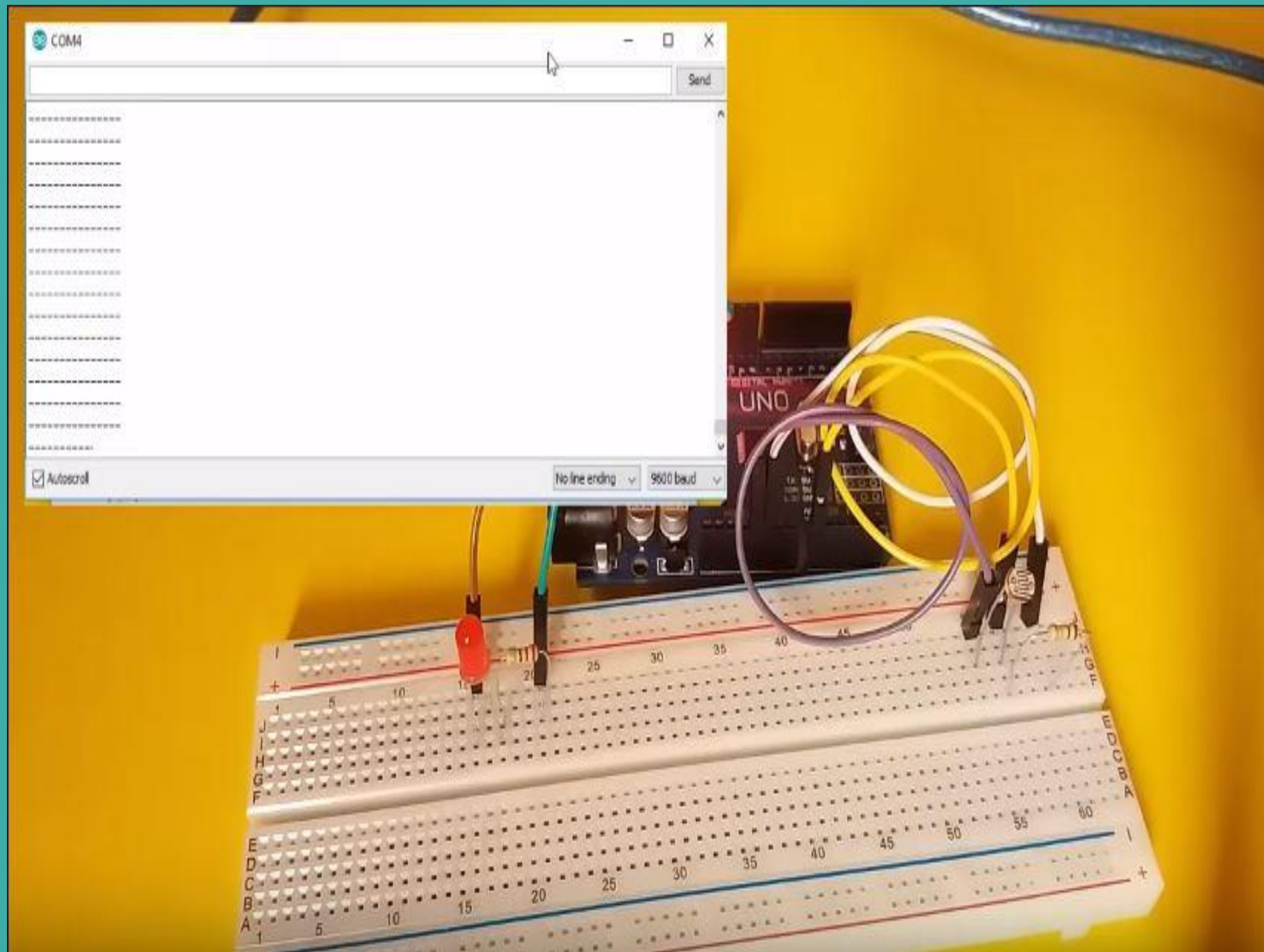
Step 2: Circuit



Step 3: IDE Code

```
const int ledPin = 13;
const int ldrPin = A0;
void setup() {
    Serial.begin(9600);
    pinMode(ledPin, OUTPUT);
    pinMode(ldrPin, INPUT);
}
void loop() {
    int ldrStatus = analogRead(ldrPin);
    if (ldrStatus <=300) {
        digitalWrite(ledPin, HIGH);
        Serial.println("LDR is DARK, LED is ON");
        delay (500);
    } else {
        digitalWrite(ledPin, LOW);
        Serial.println("-----");
        delay (500);
    }
}
```

Expected Outcome



<https://www.youtube.com/watch?v=4fN1aJMH9mM>